Improvements to Backward Proof Search with Iterative Deepening and Rule Application Prioritisation

Madeline Abio

Griffith University, Gold Coast, QLD 4101, Australia

Abstract. Automated theorem proving for first-order logic can be studied through sequent calculi, where a proof is built by repeatedly decomposing a goal sequent into simpler subgoals. This report investigates backward proof search for the pedagogical LK′ calculus of Hou [4], starting with the naïve backtracking procedure of the same textbook (Algorithm 2, p. 67). The procedure is simple to implement and closely follows the calculus, but its search space is exponential: reusable quantifier rules instantiate witnesses without consuming the quantified formula, and branching connectives compound the branching factor. A Rust implementation evaluates two refinements over this baseline: iterative deepening [5], which repeatedly runs depth-bounded search to find shallow proofs before deeper distractions, and a six-class rule priority schedule that attempts closing and low-branching rules before more expansive quantifier steps. Across three datasets totalling 1,370 problems within a one-second budget, the baseline decides 239 problems and the combined priority-ID prover decides 321, giving a 34 % improvement. Iterative deepening contributes the majority of the gain by avoiding premature commitment to deep quantifier-instantiation chains; priority scheduling adds consistent but smaller benefits by delaying costly rule applications.

Keywords: Sequent calculus · LK′ · Backward proof search · Iterative deepening · First-order logic · TPTP.

1 Introduction

Automated theorem proving (ATP) for first-order logic is the problem of mechanically determining whether a formula follows from given axioms. Industrial-strength provers such as Vampire [6] and E [8] use saturation-based resolution and superposition calculi; the connection-based leanCoP [7] avoids explicit instantiation through matrix methods and unification. These systems solve thousands of problems from the TPTP library [9] and are applied in hardware verification, software model-checking, and proof-assistant automation. The pedagogical LK′ calculus of Hou [4, Fig. 2.3, p. 65] occupies a different position: it is a backward sequent search whose rules can be stated in twenty lines of pseudocode, making it suitable for formal study, but whose naïve rule-application order can lead to pathological search behaviour. LK′ eliminates the structural

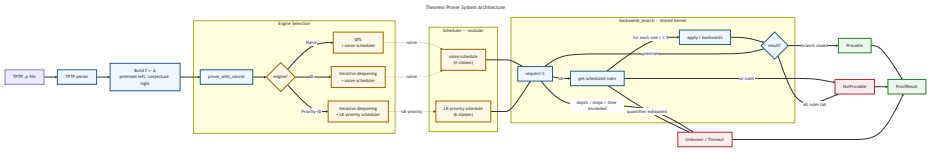

Fig. 1. System architecture of the theorem prover. Engine selection determines both the search strategy and the injected scheduler; `backwards_search` is shared across all engines.

rules of LK [2] and makes $\forall L$ and $\exists R$ *reusable*: the quantified formula is retained after instantiation so the same axiom can be applied with multiple witnesses on a single branch, and the non-deterministic choice of witness is the principal source of search-space explosion.

This paper identifies two structural failure modes in the naïve LK′ backward search of Hou [4, Algorithm 2, p. 67] and proposes two targeted improvements: a six-class *priority schedule* that refines rule-application order (Sect. 2) and an *iterative-deepening* envelope around the depth-first search [5]. An empirical evaluation on three datasets (888 TPTP problems, 280 FOLIO problems, 202 synthetic) shows consistent improvement in decided problems across all problem families, with gains concentrated where short proofs exist but are blocked by deep quantifier-instantiation chains (Sect. 4).

2 Proposed Approach

2.1 System Architecture

Figure 1 shows the end-to-end pipeline: a TPTP-style problem is parsed into a sequent $\Gamma \vdash \Delta$, dispatched to one of three engines, each of which injects a scheduler policy into the shared `backwards_search` kernel and optionally wraps it in an iterative-deepening envelope. All three engines share every component except the injected scheduler. The kernel returns one of two *decisions*: *Provable*, or *NotProvable* when every branch is closed or unclosable. If resources are exhausted before deciding, it returns *Unknown* (depth, step, or witness budget reached) or *Timeout* (wall-clock limit reached) instead. A problem is *decided* when the prover returns *Provable* or *NotProvable*; first-order logic is in general undecidable [1], so a sound procedure must return *Unknown* rather than *NotProvable* when a resource bound is reached without closing every branch.

2.2 Baseline: Naive Backward Search

Backward proof search applies LK′ inference rules *root-first*: the prover starts from the goal sequent $\vdash A$ and decomposes it until every open leaf is closed by a *closing rule* (id, $\top R$, or $\bot L$). Branching rules ($\wedge R$, $\vee L$, $\rightarrow L$) spawn multiple sub-goals that must all close; non-branching rules produce a single successor.

Quantifier rules divide into *eigenvariable* rules ($\forall R$, $\exists L$), which substitute a fresh constant not appearing elsewhere in the sequent, and *reusable* rules ($\forall L$, $\exists R$), in which the original quantified formula is retained after instantiation, permitting repeated application with different witness terms.

The naïve procedure of Hou [4, Algorithm 2, p. 67] applies rules in five mutually-exclusive priority groups: (1) closing rules; (2) invertible propositional rules together with eigenvariable quantifiers; (3) branching propositional rules; (4) reusable quantifiers with a visible witness not yet used; (5) reusable quantifiers with a freshly generated witness. The naïve engine makes a single pass through `backwards_search` with no outer loop (Algorithm 1, Appendix A). Exhausting the per-occurrence fresh-witness budget for group 5 returns *Unknown*, preserving soundness.

2.3 Failure Modes

We identify two structural weaknesses of Algorithm 1.

(F1) Eager eigenconstants. Group 2 lumps $\forall R/\exists L$ with invertible propositional rules, so the prover introduces a fresh eigenconstant before branching can close the branch propositionally, unnecessarily enlarging the witness pool.

(F2) Unbounded reusable instantiation. Groups 4–5 permit an arbitrary number of witness terms before backtracking, so a naïve DFS commits to deep instantiation chains before revisiting shallower alternatives.

Backward proof search has exponential worst-case complexity: the search is an AND-OR tree in which propositional splitting creates AND-branches and witness selection creates OR-branches, with the OR-branching factor growing under (F2) as each fresh witness enlarges the visible term pool.

2.4 Proposed Algorithm

Two targeted refinements address (F1) and (F2) while leaving the shared search kernel unchanged.

Building Block A: Iterative Deepening Our first refinement wraps the DFS in an iterative-deepening envelope [5]. The outer loop (Algorithm 2, Appendix A) retries `backwards_search` at increasing depth limits $d = 1, 2, \ldots, D$, returning as soon as some depth yields a decision: every branch is closed (Provable), or every branch is closed or unclosable within the depth bound (NotProvable). Every proof of depth k is therefore found before any branch of depth $k{+}1$ is explored, directly mitigating (F2).

Building Block B: LK′ Six-Class Priority Schedule Our second refinement replaces the five-group scheduler of Algorithm 1 with a six-class schedule (PrioritySchedule, Algorithm 2, Appendix A). The classes are applied in order, returning the first non-empty set of applicable rule instances:

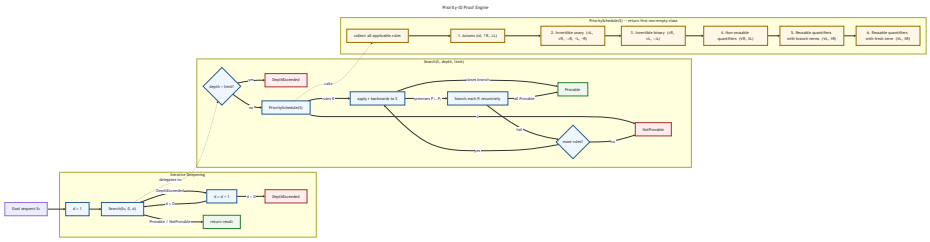

Fig. 2. Priority-ID proof engine: iterative-deepening envelope around `backwards_search` with the six-class LK′ priority scheduler.

1. **Closing rules** (id, $\top R$, $\bot L$): terminate the branch; no new sub-goals.
2. **Non-branching propositional** ($\wedge L$, $\vee R$, $\to R$, $\neg L$, $\neg R$): invertible, decompose without splitting.
3. **Branching propositional** ($\wedge R$, $\vee L$, $\to L$): split into sub-goals without enlarging the term universe.
4. **Eigenvariable quantifiers** ($\forall R$, $\exists L$): introduce a fresh eigenconstant, deferred until after branching to address (F1).
5. **Reusable quantifiers, visible witnesses** ($\forall L$, $\exists R$ with a term already on the branch): finite class exhausted before generating new symbols.
6. **Reusable quantifiers, fresh fallback** ($\forall L$, $\exists R$ with a fresh witness, subject to a per-occurrence budget): budget exhaustion returns *Unknown*, preserving soundness.

Splitting the original groups 4–5 of Algorithm 1 into classes 5–6 means the prover always exhausts visible witnesses before introducing a new symbol, addressing (F2).

Combined Prover. The proposed prover `priority-id` combines both blocks (Figure 2); two control engines isolate their contributions: `id` (iterative deepening with the naïve scheduler) and `naive` (the unmodified baseline).

2.5 Expected Performance

`id` should account for the majority of additional decisions: by guaranteeing that every proof of depth k is discovered on iteration k, iterative deepening stops the naïve DFS from committing to deep quantifier-instantiation chains before shallower alternatives are exhausted, directly mitigating (F2). The six-class priority schedule then yields a smaller, consistent gain by firing closing and invertible rules before any AND-split, and by exhausting visible witnesses before introducing a fresh symbol, addressing (F1).

3 Implementation

The prover is written in Rust [18], using `pest` [15] for FOF parsing, `clap` [11] for the CLI, `rusqlite` [17] for result persistence, and `ctrlc` [12] for cancellation.

Per-problem parallel evaluation uses `rayon` [16]; `log`/`env_logger` [14,13] provide structured logging and `chrono` [10] timestamps SQLite rows. The three engines share all code outside the rule scheduler (Figure 1). All benchmarks ran on Windows 11 Home (Intel Core Ultra 7 258V, 32 GB RAM, Rust 1.85) with a 1,000 ms wall-clock timeout, recursive depth 128, and a per-occurrence fresh-fallback budget of one term; step limits were 10,000 for TPTP and FOLIO, and effectively unlimited (10^{11}) for synthetic, because raising the step limits for TPTP and FOLIO did not meaningfully change the outcome, whereas it did for the synthetic data. Problems with more than six biconditionals were skipped before parsing due to their computational complexity. Results were persisted to SQLite and analysed with Python 3.12.

4 Experiments

4.1 Datasets

TPTP. Two subsets of the TPTP problem library [9] are used. The first is a 737-problem provable subset (FOF/THM), restricted to problems with 0–50 input formulae, 0–150 atoms, and no equality or arithmetic. The second is a 204-problem counter-satisfiable subset (FOF/CSA) drawn from the same complexity bounds, in which each problem is known to have no proof because its negation is satisfiable. The two subsets together test both sides of the prover against problems with known status across diverse mathematical domains.

FOLIO. The FOLIO dataset [3] provides 280 human-authored natural-language reasoning problems formalised as FOF, testing whether the prover's behaviour generalises beyond the controlled TPTP filter.

Synthetic. 202 synthetic FOF problems were generated programmatically across four tiers, *easy* (32), *medium* (60), *hard* (60), *expert* (50), with proof depth $n \in [2, 15]$ and distractor count $d \in [0, 14]$ scaling jointly with tier. Four problem families are used. **Implication chains**: a conjecture reachable from a ground fact via n universally quantified steps, with d structurally identical distractor axioms. **Syllogisms**: a fixed class hierarchy with a ground instance for one individual. **Transitivity chains**: a single binary transitivity axiom plus ground edge facts, requiring repeated reuse of the same axiom across different instantiations. **Quantifier alternation**: single-formula conjectures testing classical $\forall/\exists$ theorems and non-theorems. Unprovable variants negate the conjecture (e.g., remove a chain link, change the syllogism individual, or ask for an unreachable node).

4.2 Results

Table 1 reports decided counts across all datasets.

Table 1. Decided problems per engine (timeout 1,000 ms, depth 128, fresh-fallback 1; steps 10,000 for TPTP/FOLIO, unlimited for synthetic). *Decided*: prover returns *Provable* or *NotProvable*.

Engine	TPTP (888)	FOLIO (280)	Synthetic (202)	Total (1370)
`naive`	156 (17.6 %)	57 (20.4 %)	26 (12.9 %)	239 (17.4 %)
`id`	203 (22.9 %)	57 (20.4 %)	39 (19.3 %)	299 (21.8 %)
`priority-id`	213 (24.0 %)	66 (23.6 %)	42 (20.8 %)	321 (23.4 %)

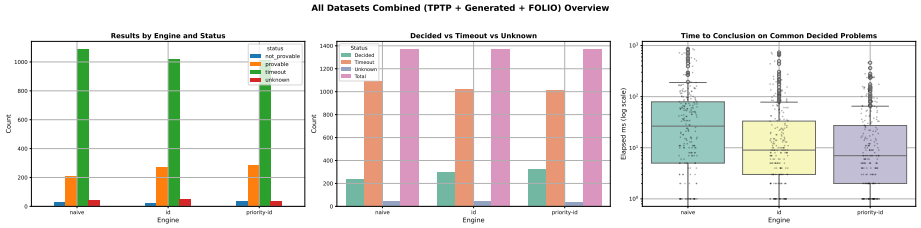

Fig. 3. Outcomes by engine across all three datasets (TPTP 888, FOLIO 280, synthetic 202). For each dataset: outcome counts by engine (left), decided vs. timeout vs. unknown breakdown (centre), and time to conclusion on commonly decided problems, log scale (right). Per-dataset views in Figs. 4, 5, and 6, Appendix C.

On TPTP (Fig. 4, Appendix C), `naive` decides 156, `id` 203 (+30 %), and `priority-id` 213 (+37 %); all gains come from the provable subset, as the four counter-satisfiable decisions are shared. `id` and `priority-id` also solve shared problems faster on average, consistent with fewer wasted steps in unclosable branches.

The FOLIO results (Fig. 5, Appendix C) reveal a different pattern: `naive` and `id` both decide 57 problems but with shifted composition (`naive` 34/23 *Provable*/*NotProvable*; `id` 40/17). Iterative deepening converts 11 naive-timeouts to *Provable* but offsets this by causing 5 *Provable* decisions to time out and 6 *NotProvable* decisions to become *Unknown*/*Timeout*. `priority-id` resolves the trade-off, deciding 66 problems (38/28) by converting 11 of `id`'s *Unknown*/*Timeout* outcomes to *NotProvable* and reducing *Unknown* from 25 to 10.

On the synthetic dataset (Fig. 6, Appendix C), `naive` decides 26, `id` 39 (+50 %), and `priority-id` 42 (+62 %). Gains concentrate in the medium and hard tiers (Table 5, Appendix B); no engine decides any expert-tier problem, and all 43 transitivity-chain problems time out under every engine. Figure 9 (Appendix C) confirms `naive`'s solve rate collapses beyond depth 4 while `id` and `priority-id` maintain non-zero rates into the hard tier: the signature of (F2).

5 Discussion

The shared-kernel design (Sect. 2.4) lets us attribute the 82 gained decisions over the baseline to specific failure modes rather than to engine identity. Iterative

deepening alone accounts for 60 of those 82 (73 %), implicating (F2), unbounded reusable instantiation, as the dominant bottleneck on these datasets. The priority schedule contributes the remaining 22 (27 %) on top of `id`, so (F1) is real but secondary in volume.

FOLIO complicates an additive reading. Iterative deepening alone ties the baseline (57 = 57): every *Provable* it gains is offset by another decision regressing. Adding the priority schedule on top recovers the regressions and converts 11 of `id`'s *Unknown*/*Timeout* outcomes into *NotProvable*. The two refinements are therefore not independent; depth bounds and rule scheduling interact, and iterative deepening's net benefit is contingent on the rule schedule it runs against.

The synthetic results expose where these refinements stop helping. All 43 transitivity-chain problems time out under every engine, because reusing one axiom across many witnesses defeats both depth-bounded search and priority ordering. The expert tier is undecided across the board, echoing TPTP behaviour above rating 0.10: the same structural weakness limits both real and synthetic input at the hard end.

Roughly 70 % of problems remain undecided overall, but the residue is dominated by *Timeout* rather than *Unknown*; the implementation is not blocked by its resource bounds, it is blocked by the rate at which the search space grows under (F2). A wider envelope (longer timeouts, deeper recursion) would buy less than a structural change that prunes redundant witness instantiation, such as lemma sharing or a connection-based kernel.

6 Conclusion

Small, principled changes to rule-application order produce disproportionate improvements in practical solve rate: iterative deepening and the six-class LK′ priority schedule together raise decided problems from 239 to 321 across three diverse datasets without altering the proof calculus. Key limitations: the fast solve times offered by rust are non-determinstic due to OS jitter; no TPTP problem rated above 0.10 is solved, confining the prover to the easiest tier; direct Rust recursion bounds `max_depth` at the stack limit; and the prover is not competitive with mature systems such as leanCoP [7] or Vampire [6]. Natural successors are lemma sharing (caching closed subgoals) and a transition to connection-based or saturation-based proof strategies [6,8].

6.1 Data Availability.

Source code and the SQLite databases backing every figure in this paper are available at https://github.com/madiabio/automated-first-order-logic-prover.

References

1. Church, A.: A note on the Entscheidungsproblem. The Journal of Symbolic Logic **1**(1), 40–41 (1936)

2. Gentzen, G.: Untersuchungen über das logische schließen. ii. Mathematische Zeitschrift **39**(1), 405–431 (1935). https://doi.org/10.1007/BF01201363, https://doi.org/10.1007/BF01201363
3. Han, S., et al.: FOLIO: Natural language reasoning with first-order logic. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. pp. 22017–22031. Association for Computational Linguistics (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1229
4. Hou, Z.: Fundamentals of Logic and Computation. Texts in Computer Science, Springer, Cham (2021). https://doi.org/10.1007/978-3-030-87882-5, https://doi.org/10.1007/978-3-030-87882-5
5. Korf, R.E.: Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence **27**(1), 97–109 (1985). https://doi.org/10.1016/0004-3702(85)90084-0
6. Kovács, L., Voronkov, A.: First-order theorem proving and vampire. In: Sharygina, N., Veith, H. (eds.) Computer Aided Verification. pp. 1–35. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
7. Otten, J., Bibel, W.: leancop: Lean connection-based theorem proving. Journal of Symbolic Computation Submitted
8. Schulz, S.: E – a brainiac theorem prover. AI Communications **15**(2-3), 111–126 (2002). https://doi.org/10.3233/EAI-2002-260
9. Sutcliffe, G.: The TPTP Problem Library and Associated Infrastructure. From CNF to TH0, TPTP v6.4.0. Journal of Automated Reasoning **59**(4), 483–502 (2017). https://doi.org/10.1007/s10817-017-9407-7
10. The chrono Contributors: chrono: Date and time library for Rust. https://crates.io/crates/chrono (2024)
11. The clap Contributors: clap: Command line argument parser for Rust. https://crates.io/crates/clap (2024)
12. The ctrlc Contributors: ctrlc: Easy Ctrl-C handler for Rust programs. https://crates.io/crates/ctrlc (2024)
13. The env_logger Contributors: env_logger: A logging implementation for log configured via env vars. https://crates.io/crates/env_logger (2024)
14. The log Contributors: log: A lightweight logging facade for Rust. https://crates.io/crates/log (2024)
15. The pest Contributors: pest: The elegant parser. https://pest.rs (2024)
16. The rayon Contributors: rayon: A data parallelism library for Rust. https://crates.io/crates/rayon (2024)
17. The rusqlite Contributors: rusqlite: Ergonomic bindings to SQLite for Rust. https://crates.io/crates/rusqlite (2024)
18. The Rust Programming Language Contributors: The rust programming language. https://www.rust-lang.org (2024), edition 2024

A Algorithms

Algorithm 1 is the naïve backward proof-search procedure of Hou [4, Algorithm 2, p. 67], rewritten in the notation used throughout this paper; Algorithm 2 is the full priority-ID pseudocode.

Algorithm 1 Naïve backward proof search for LK′ (Algorithm 2 of [4, p. 67])

Require: First-order formula A
Ensure: Derivation tree in LK′, or report no proof found
1: build the bottom sequent $\vdash A$
2: **for all** top sequents on an open branch **do**
3: **if** any of $id, \top R, \bot L$ is applicable **then**
4: apply the rule backwards and close the branch
5: **else if** any of $\wedge L, \vee R, \to R, \neg L, \neg R, \forall R, \exists L$ is applicable **then**
6: apply the rule backwards
7: **else if** any of $\wedge R, \vee L, \to L$ is applicable **then**
8: apply the rule backwards and create a new branch
9: **else if** $\forall L$ or $\exists R$ is applicable, and there is a term t that has not been used to instantiate the quantified variable x in this formula **then**
10: apply the rule backwards, substituting x with t
11: **else if** $\forall L$ or $\exists R$ is applicable **then**
12: apply the rule backwards and create a fresh term
13: **else**
14: stop
15: **end if**
16: **end for**

Algorithm 2 Priority-ID backward proof search for LK′

Require: Goal sequent S_0, maximum depth D
Ensure: Provable, NotProvable, or DepthExceeded

for $d \leftarrow 1$ **to** D **do** ▷ try increasing depth budgets
  $result \leftarrow \textsc{Search}(S_0,\ 0,\ d)$
  if $result \neq \textsc{DepthExceeded}$ **then**
    return $result$
  end if
end for
return DepthExceeded
function $\textsc{Search}(S,\ depth,\ limit)$
  if $depth > limit$ **then**
    return DepthExceeded
  end if
  $R \leftarrow \textsc{PrioritySchedule}(S)$
  if $R = \emptyset$ **then**
    return NotProvable ▷ no rule applies
  end if
  for all rule $r \in R$ **do**
    $premises \leftarrow$ apply r backwards to S
    if $premises = \emptyset$ **then** ▷ axiom: branch closed immediately
      return Provable
    end if
    if $\forall P \in premises : \textsc{Search}(P,\ depth + 1,\ limit) = \textsc{Provable}$ **then**
      return Provable
    end if
  end for
  return NotProvable
end function
function $\textsc{PrioritySchedule}(S)$
  Collect all applicable LK′ rule instances from S.
  Return the **first non-empty** class from the ordered list below.

  1. Axioms: $id,\ \top R,\ \bot L$
  2. Invertible, non-branching: $\wedge L,\ \vee R,\ \to R,\ \neg L,\ \neg R$
  3. Invertible, branching: $\wedge R,\ \vee L,\ \to L$
  4. Non-reusable quantifiers: $\forall R,\ \exists L$
  5. Reusable quantifiers ($\forall L,\ \exists R$) with terms already on the branch
  6. Reusable quantifiers ($\forall L,\ \exists R$) with one new fresh term

  return $\emptyset$ if no class has a match.
end function

B Per-dataset outcome tables

Tables 2–4 give the full outcome breakdown for each dataset and engine. Outcomes are *Provable*, *NotProvable* (NP), *Timeout* (TO), and *Unknown* (U). Table 5 gives the per-tier decided counts for the synthetic dataset.

Table 2. TPTP provable subset (737 problems).

Engine	Provable	TO	U	Total
`naive`	152	573	12	737
`id`	199	526	12	737
`priority-id`	209	516	12	737

Table 3. TPTP counter-satisfiable subset (151 problems).

Engine	NP	TO	U	Total
`naive`	4	143	4	151
`id`	4	143	4	151
`priority-id`	4	142	5	151

Table 4. Synthetic dataset (202 problems).

Engine	Provable	NP	TO	U	Total
`naive`	22	4	168	8	202
`id`	35	4	155	8	202
`priority-id`	38	4	152	8	202

Table 5. Synthetic dataset: decided problems per engine by difficulty tier.

Engine	Easy (32)	Medium (60)	Hard (60)	Expert (50)
`naive`	16	10	0	0
`id`	17	19	3	0
`priority-id`	17	20	5	0

C Auxiliary figures

Figures 4–8 give per-dataset breakdowns of the combined overview (Fig. 3); Figure 9 gives the decision-rate vs. chain-depth plot for implication-chain problems; Figure 10 gives the tier breakdown for the synthetic dataset; Figure 11 shows the baseline engine diagram.

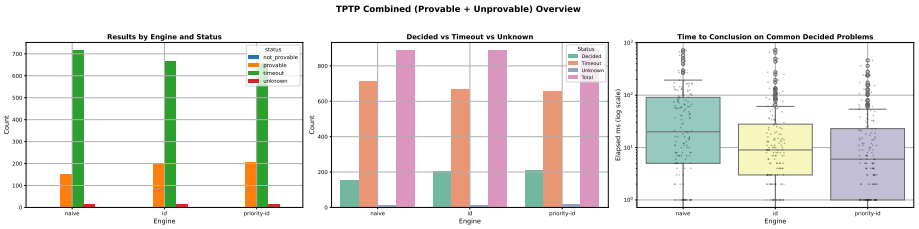

Fig. 4. TPTP combined dataset (888 problems): outcome counts by engine (left), decided vs. timeout vs. unknown breakdown (centre), and time to conclusion on commonly decided problems, log scale (right).

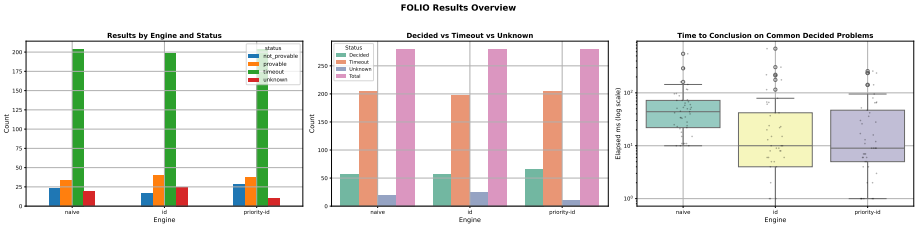

Fig. 5. FOLIO validation set (280 problems): outcome counts by engine (left), decided vs. timeout vs. unknown breakdown (centre), and time to conclusion on commonly decided problems, log scale (right).

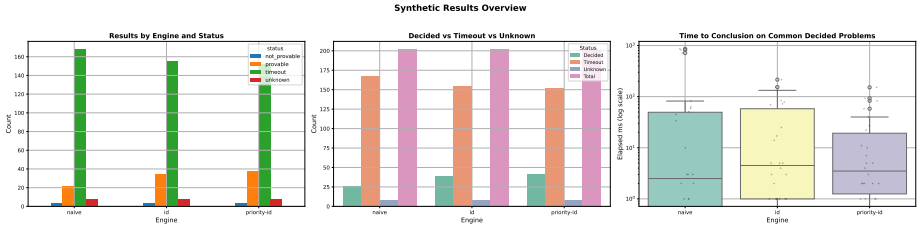

Fig. 6. Synthetic dataset (202 problems): outcome counts by engine (left), decided vs. timeout vs. unknown breakdown (centre), and time to conclusion on commonly decided problems, log scale (right).

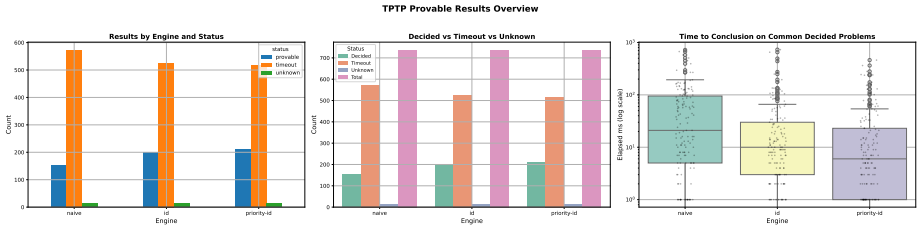

Fig. 7. TPTP provable subset (737 problems): outcome counts by engine, decided vs. timeout vs. unknown, and timing. See Table 2 for numerical breakdown.

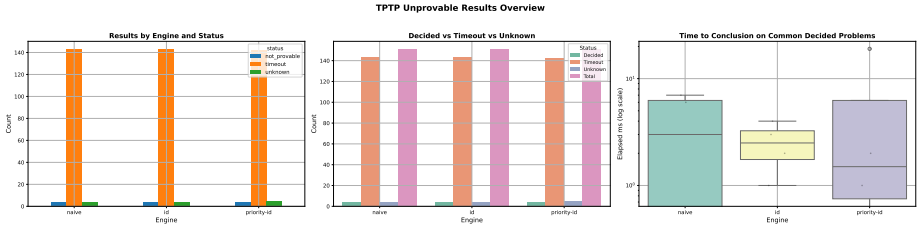

Fig. 8. TPTP counter-satisfiable subset (151 problems): outcome counts by engine, decided vs. timeout vs. unknown, and timing. See Table 3 for numerical breakdown.

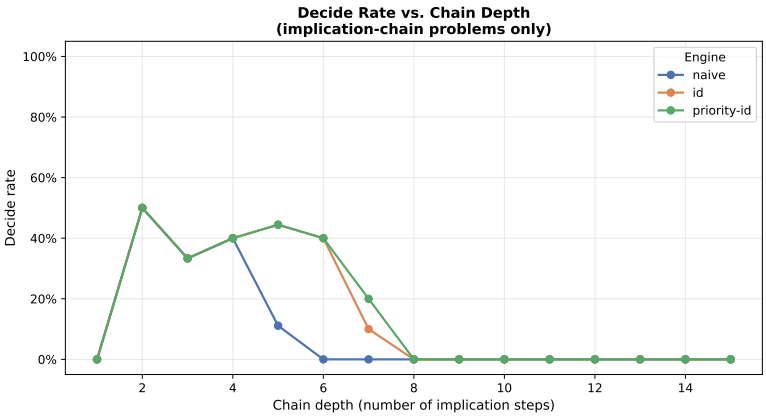

Fig. 9. Decision rate vs. chain depth for implication-chain problems (fraction decided per depth, by engine).

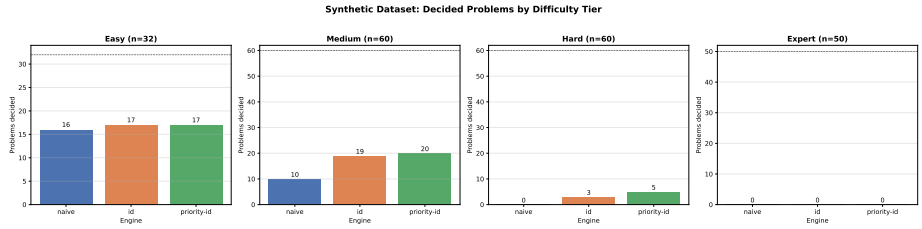

Fig. 10. Synthetic dataset (202 problems): decided problems per engine at each difficulty tier. The dashed line marks the total number of problems in that tier.

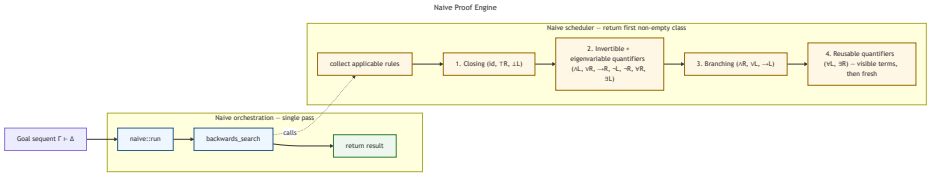

Fig. 11. Naive proof engine: single-pass DFS with the five-group naive scheduler.

D Generative AI Use

Two AI tools were used heavily for code development. Where possible, queries are reproduced from session logs with minor typographical corrections for readability but are otherwise verbatim; those corrections were themselves made with Claude Sonnet 4.6 assistance.

Claude Sonnet 4.6 (Anthropic), accessed via the Claude Code command-line interface and web interface (`claude.ai/code`), was the primary tool. All Claude-assisted commits carry a `Co-Authored-By: Claude Sonnet 4.6` trailer that Claude Code automatically appends, providing verifiable attribution in the git history. The following queries and responses are reproduced from local Claude Code session logs (`.jsonl` files stored by the CLI); the priority-scheduler session (§D) was conducted via the web interface and is documented from commit trailers only, as no local log was retained.

OpenAI Codex was also used during development for feature implementation, design discussion, and code-quality passes. Session logs were not retained for Codex interactions and Codex does not automatically tag commits, so specific queries cannot be reproduced.

A.2.1 Iterative Deepening Engine (6 May 2026)

Queries:

> I want to implement an iterative deepening backwards search engine. The feature may not need to be a full engine, but the idea is to use the clap CLI args so the user can select via `-engine naive` or `-engine id`. I will be adding more optimisations later, but this is the starting point. The current architecture should support iterative deepening. Here is the algorithm: [Algorithm 2, p. 67, Hou 2021, supplied as full LaTeX pseudocode].
>
> I started an ultraplan session in the web browser and it began editing code. I would like to continue from the CLI instead, using this plan: [Plan: Iterative Deepening Backward Search Engine — `-engine` flag, `ProofOptions::engine` field, ID loop calling `backwards_search` at increasing depth limits, `engine/id.rs` module].
>
> Please also make sure the engine can be configured in `config.toml`, that everything is documented appropriately, and that all relevant functions have appropriate rustdocs.

Outcome: Claude implemented the `-engine naive|id` CLI flag, `ProofOptions::engine` field, and the iterative-deepening loop in a new `engine/id.rs` submodule, adding rustdocs throughout (commits `d16bb3c` and associated engine-modularisation commits).

A.2.2 SQLite Persistence (6 May 2026)

Queries:

> I need to make the results from each run for each engine persist so I can analyse the data. How should I do this?
>
> SQLite would be preferable — can we use that instead?
>
> The interface should be something like `-persist true` or `-persist false`, defaulting to `true`, with the database location configurable — perhaps `-persist <DB_FILE_LOCATION>` with a default path specified in `config.toml`.
>
> The CLI flag wiring should live in `cli/`, but the actual insertion logic should be in its own module such as `src/persistence`. What is your recommendation?

Outcome: Claude designed and implemented the `src/persistence/` module with an SQLite schema, `-persist` CLI flag with configurable path, and a TDD test suite (commits `a8333b8`, `c5bc21f`, `5719e21`, `ba06ddf`, `2ca6f0e`, `a1de674`).

A.2.3 Six-Class Priority Scheduler (6 May 2026)

Note: This session was conducted via the Claude Code web interface; no local session log is available. The session is documented from the `Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>` trailers that Claude Code automatically appends to commits it helps author.

Outcome: Claude implemented the six-class LK′ rule priority scheduler (closing rules; non-branching propositional; branching propositional; eigenvariable quantifiers; reusable quantifiers with visible witnesses; reusable quantifiers with fresh fallback) and the `Priority` and `PriorityId` engine variants (commits `8d72635`, `663ac9a`, `d16bb3c`).

A.2.4 FOLIO Rust Integration (9 May 2026)

Queries:

> I am happy to modify the prover so that it does not require a `.p` file — it can be adapted to accept whatever format is needed. Please have a look at `theorem_prover/` to determine the best approach.
>
> Ensure every new function is appropriately documented with rustdocs, and maintain appropriate separation of concerns across modules, files, and functions. Create new submodules or files where needed to meet good software design standards.

Outcome: Claude extended the grammar and parser to accept FOLIO `.jsonl` problems directly without converting to TPTP `.p` format, adding a new parser module with full rustdoc coverage.

A.2.5 Parallelisation and Channel-Based DB Writer (9 May 2026)

Queries:

> I would like to run problem files in parallel within `theorem_prover/`. I am considering migrating persistence from SQLite to PostgreSQL to support this, though I am not sure that is the right approach.
>
> For parallelisation: multiple files simultaneously within a single invocation.
>
> Out-of-order progress output is acceptable. Please implement this with TDD. The timer logic can also be simplified, and the Ctrl-C handling can likely be simplified now that the owner thread can terminate the workers directly.

Outcome: Claude implemented `rayon par_iter` over problem paths with an `AtomicUsize` progress counter, a channel-based SQLite writer thread to avoid connection-sharing across threads, and an exit-code fix (`process::exit` replaced with a return value to ensure the writer thread flushes before exit) (commits `c52a7bf`, `95cc45a`, `d48cc7b`).